

## 说明

SPI (Serial Peripheral Interface) 是 MOTOROLA 提出的四线同步串行协议, 广泛用于连接 LCD 屏幕、FLASH、ADC 及其他外设等。SPI 外设则是为了 SPI 协议而设计的外设, 具有速度快, 灵活性高等特点。而 QSPI 外设则是专门用于兼容用于连接支持四线 (Quad) 传输模式的 SPI 闪存 (如 Quad-SPI NOR Flash) 等设备的增强型外设。

### 适用范围

| 适用范围   | 系列                  |
|--------|---------------------|
| 通用 MCU | CH32H417 CH32V205 等 |

# 目录

|                              |   |
|------------------------------|---|
| 说明 .....                     | 1 |
| 目录 .....                     | 2 |
| 表格索引 .....                   | 2 |
| 图片索引 .....                   | 2 |
| 第 1 章 SPI 与 QSPI .....       | 1 |
| 1.1 SPI 的简介 .....            | 1 |
| 1.2 QSPI 的介绍 .....           | 1 |
| 1.2.1 可编程命令帧 .....           | 1 |
| 1.2.2 功能模式 .....             | 1 |
| 1.2.3 XIP 能力 .....           | 2 |
| 1.2.4 QSPI 的功能框图 .....       | 2 |
| 1.3 SPI 与 QSPI 的对比 .....     | 3 |
| 第 2 章 QSPI 读取 FLASH 例程 ..... | 4 |
| 2.1 QSPI 初始化 .....           | 4 |
| 2.2 QSPI 进行读取 .....          | 4 |
| 2.3 QSPI 进行写入 .....          | 5 |
| 2.4 QSPI 轮询状态 .....          | 6 |
| 历史版本 .....                   | 8 |
| 声明 .....                     | 9 |

# 表格索引

|                     |   |
|---------------------|---|
| 帧配置 .....           | 1 |
| SPI 与 QSPI 对比 ..... | 3 |

# 图片索引

|               |   |
|---------------|---|
| QSPI 框图 ..... | 2 |
|---------------|---|

## 第1章 SPI 与 QSPI

### 1.1 SPI 的简介

SPI (Serial Peripheral Interface) 是四线同步串行协议，需要引脚 MISO、MOSI、SCK、NSS。时钟 (SCK) 始终由主机产生，数据以全双工方式在 MOSI/MISO 上同拍收发；MISO/MOSI 在主从模式下的方向互换 (主模式 MOSI 输出、MISO 输入，从模式反之)。

一般来说，SPI 外设的使用比较灵活，用户可以选择字长 (8/16 位)、位序 (MSB/LSB 在前) 以及四种时钟模式 (由 CPOL 和 CPHA 组合决定)；在片选管理上，既支持通过软件配合 GPIO 灵活控制多个从机，也支持硬件自动管理 NSS。

这样使得 SPI 外设可以支持几乎所有 SPI 设备的需要。

### 1.2 QSPI 的介绍

QSPI (QuadSPI) 是目标用于连接 SPI FLASH 存储介质的主控制器。但是也可以连接其他兼容其协议的设备，如 PSRAM，LCD 屏幕等。相对 SPI 外设来说，QSPI 的灵活性没有 SPI 高，但是 QSPI 可以同时使用多个 IO 进行传输，使得 QSPI 有更高的带宽。

QSPI 使用的引脚为 SCK、SCSN、SI00/SI01/SI02/SI03，其中 SI00-SI03 是双向复用数据线，可单独组态为 1/2/4 线传输；不用 4 线的引脚可释放作其它用途。

#### 1.2.1 可编程命令帧

QSPI 在通信时，每次交互是一条命令，至多由五个阶段组成，任一阶段都可跳过，但是至少需要保留一个阶段，各阶段的线数 (1/2/4) 独立可配；对应的流程如下：

nCS↓ | 指令 | 地址 | 交替字节 | 空指令 | 数据 | nCS↑

对应 CCR 寄存器的字段：

表 1-1 帧配置

| 阶段   | 对应可配置内容                      | 配置内容                              |
|------|------------------------------|-----------------------------------|
| 指令   | INSTRUCTION、IMODE            | 8 位操作码 (如 0x03 读、0x02 页写)；1/2/4 线 |
| 地址   | AR. ADDRESS、ADSIZE、ADMODE    | 1~4 字节地址；1/2/4 线/无                |
| 交替字节 | ABR. ALTERNATE、ABSIZE、ABMODE | 地址后紧跟的控制字节 (不常用)                  |
| 空指令  | DCYC                         | 空闲的 SCK 周期，供设备准备数据                |
| 数据   | DR/DLR、DMODE[1:0]            | 任意长度数据，经 32 字节 FIFO 收发；1/2/4 线/无  |

IMODE/ADMODE/ABMODE/DMODE = 00b 即跳过对应阶段；DummyCycles = 0 略过空指令。整条帧由硬件组合完成，软件只需写各组成寄存器并给出最后一项自行开启。

#### 1.2.2 功能模式

QSPI 在配置命令时可以选择多种功能模式：

- 间接模式 (00 写 / 01 读): 数据经 DR 寄存器 + 32 字节 FIFO 手动或 DMA 搬移, DLR 指定字节数。适合频繁的读写。
- 自动轮询模式 (10): 硬件周期性启动命令读取 FLASH 状态寄存器, 按 PSMKR 屏蔽、与 PSMAR 匹配, 命中即置 SMF 并可选中断。在实际使用中用于"等擦除/写周期结束"这类忙等待, 替代软件循环读 SR。
- 内存映射模式 (11): FLASH 映射进 CPU 地址空间, 当作内部存储器直接读 (含 XIP 取指); FIFO 作预取缓冲, 可开 TCEN 超时在空闲后释放 nCS 降功耗。仅支持读。

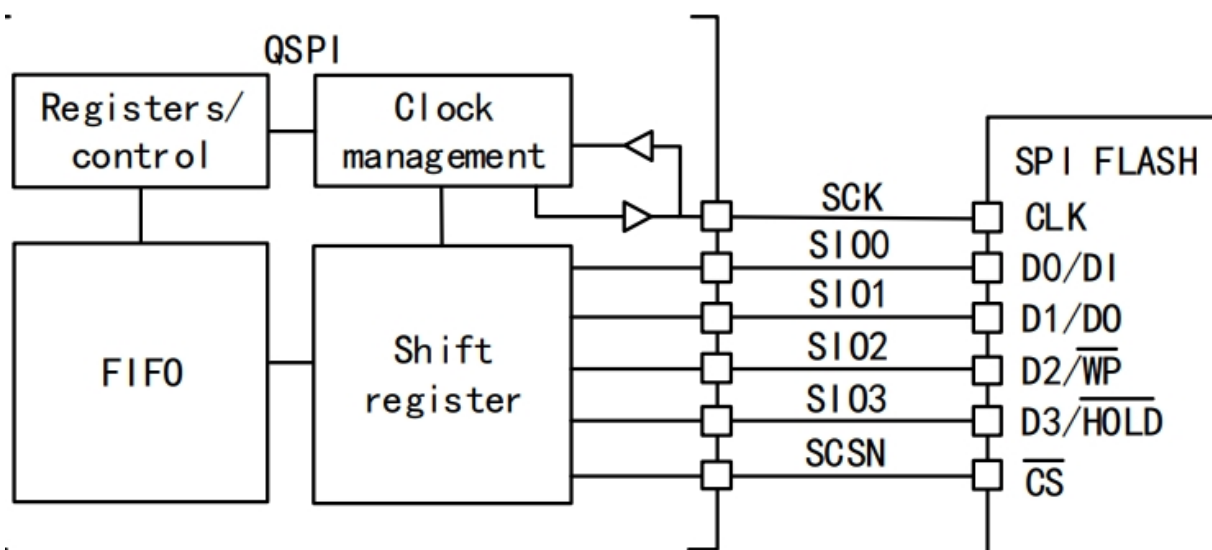
### 1.2.3 XIP 能力

QSPI 通过配置寄存器 (CCR.FMODE=11b) 使能内存映射模式, 在此 XIP 模式下, QSPI 硬件将会接管总线中对应的一片地址区域——片上总线一旦命中该区域, QSPI 就会在访问时自动发起一次针对 SPI FLASH 的读帧, 把片选拉低、地址、空指令、按需读数据的整个过程全部交由硬件组帧完成, 软件无需逐字节操作寄存器, 只需把这片区域当作普通只读存储器直接读即可; 内部 32 字节 FIFO 充当预取缓冲以吸收连续读的吞吐, 并可配置 TCEN 在空闲超时后自动拉高 nCS 以降低功耗。但需要注意

- 仅支持读: XIP/内存映射模式下 FLASH 只能当只读存储区直读, 写、擦除等仍需切回间接模式 (FMODE=00/01) 操作。
- 时序完全硬件化: CCR (指令/线数)、AR (地址)、DCYC (空指令)、FIFO 阈值等配置一次后即生效, 后续所有命中均按同一帧自动执行。
- 配置前提: DCR.FSIZE 声明的容量范围必须覆盖被访问的地址, 否则命中超界地址会置 TEF 标志。

### 1.2.4 QSPI 的功能框图

图 1-1 QSPI 框图



### 1.3 SPI 与 QSPI 的对比

表 1-2 SPI 与 QSPI 对比

|       | SPI                  | QSPI                                      |
|-------|----------------------|-------------------------------------------|
| 主从配置  | 通用收发器，可主可从           | 仅主，面向 SPI FLASH 存储介质                      |
| 数据线   | MISO / MOSI 方向固定     | SI00-3 双向复用，1/2/4 线可配                     |
| 全双工   | 支持                   | 否，读/写分时单向                                 |
| 帧组织   | 由用户自定义               | 五阶段（指令/地址/交替/空指令/数据）全可编程，集成地址与空指令         |
| nCS   | 软件 GPIO 或硬件 NSS，支持多从 | 硬件 CS                                     |
| 时钟模式  | CPOL/CPHA 四种模式       | 仅模式 0 / 模式 3（CKMODE），可等价 SPI 的模式 0 / 模式 3 |
| 状态等待  | 软件循环读状态寄存器判 BUSY     | 硬件自动轮询，掩码+匹配命中置 SMF 或中断                   |
| 内存映射  | 无                    | 支持，FLASH 映射入地址空间直接读 / XIP                 |
| 最高读吞吐 | 1 × SCK              | 单线同 SPI；4 线 = 4 × SCK                     |
| 双闪存   | 无                    | DFM 双片并行，吞吐/容量翻倍                          |
| 典型负载  | 传感器、ADC/DAC、小容量存储    | 大容量 NOR/NAND、代码存储、XIP                     |

## 第2章 QSPI 读取 FLASH 例程

本章介绍如何使用 QSPI 替代 SPI 来操作 FLASH。

### 2.1 QSPI 初始化

配置并初始化 QSPI 外设，设置时钟分频为 2 分频、SCK 空闲时为高电平的 Mode3 模式、片选保持时间至少 8 个 SCK 周期，并将 Flash 大小设为 22（对应  $2^{23}=8\text{MB}$  容量），同时选择单闪存 FLASH1 且禁用双闪存模式，最后调用初始化函数并设置 FIFO 阈值为 4 字节。

```
QSPI_InitStructure.QSPI_Prescaler = 1;          /* 值+1 为分频: FSCK = FHCLK/2 */
QSPI_InitStructure.QSPI_CKMode    = QSPI_CKMode_Mode3; /* CKMODE=1, 空闲 SCK 高 */
QSPI_InitStructure.QSPI_CSHTime   = QSPI_CSHTime_7Cycle; /* 命令间 nCS 高电平 >= 8 个 SCK */
/* 外部容量 2^(FSIZE+1)=2^23=8MB; 地址超界置 TEF */
QSPI_InitStructure.QSPI_FSize     = 22;

QSPI_InitStructure.QSPI_FSelect   = QSPI_FSelect_1; /* 单闪存选 FLASH1 */
QSPI_InitStructure.QSPI_DFlash   = QSPI_DFlash_Disable; /* DFM=0, 不用双闪存 */
QSPI_Init(QSPI1, &QSPI_InitStructure);
QSPI_SetFIFOThreshold(QSPI1, 4); /* FTHRES: FIFO 有效字节 */
```

### 2.2 QSPI 进行读取

下面的代码用于 QSPI 通过单线读取 FLASH 中的数据，其效果与后续的 SPI 读取类似。

```
/* 指令+地址走单线，数据单线；FastRead(0x0B) 带 8 个空指令周期（DCYC=8），
 * 在中高速率下给 FLASH 数据输出稳定时间，与普通读 0x03 不完全一致。 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_IMode = QSPI_ComConfig_IMode_1Line;
QSPI_ComConfig_InitStructure.QSPI_ComConfig_ADMode = QSPI_ComConfig_ADMode_1Line;
QSPI_ComConfig_InitStructure.QSPI_ComConfig_DMode = QSPI_ComConfig_DMode_1Line;
QSPI_ComConfig_InitStructure.QSPI_ComConfig_FMode = QSPI_ComConfig_FMode_Indirect_Read; /* 间接读模式 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_ADSIZE = QSPI_ComConfig_ADSIZE_24bit; /* 地址宽度 24 位 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_Ins    = FLASH_CMD_FastRead; /* 0x0B */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_DummyCycles = 8; /* 空指令周期=8 */
QSPI_ComConfig_Init(QSPI1, &QSPI_ComConfig_InitStructure);
QSPI_SetAddress(QSPI1, addr); /* ADDRESS: 24 位目标地址 */
QSPI_SetDataLength(QSPI1, len); /* DLR: 待读字节数 */
QSPI_Rx_DMA(data, len); /* DMA, 从 DR 迁走 len 字节 */
QSPI_DMAcmd(QSPI1, ENABLE); /* DMAEN */
QSPI_Start(QSPI1);
DMA_Cmd(DMA2_Channel1, ENABLE);

while (DMA_GetFlagStatus(DMA2, DMA_FLAG_TC1) == RESET) /* 等 DMA 搬完 */
;
;
```

配置 QSPI 通信参数为指令、地址和数据全部采用单线模式，使用快速读指令 0x0B 并设置 8 个空指令周期以提供数据输出稳定时间，然后调用通信配置初始化函数，接着设置目标地址和待读取

数据长度，启动 DMA 传输并将数据从 QSPI 数据寄存器搬移到内存，使能 QSPI 的 DMA 功能并启动 QSPI 传输，同时使能 DMA 通道后等待 DMA 传输完成标志置位。这样就完成了一次读取。

下面为效果相似的 SPI 代码：

```
GPIO_WriteBit(GPIOA, GPIO_Pin_2, 0); /* 拉低片选 (nCS)，选中 FLASH 设备，开始通信 */
SPI1_ReadWriteByte(FLASH_CMD_FastRead); /* 发送快速读指令 (0x0B) */
SPI1_ReadWriteByte((u8)((ReadAddr) >> 16)); /* 发送 24 位地址的高 8 位 */
SPI1_ReadWriteByte((u8)((ReadAddr) >> 8)); /* 发送 24 位地址的中间 8 位 */
SPI1_ReadWriteByte((u8)ReadAddr); /* 发送 24 位地址的低 8 位 */
SPI1_ReadWriteByte((u8)DUMMY_BYTE); /* 发送一个空指令周期 (Dummy Byte) */

for (i = 0; i < size; i++) /* 循环读取数据 */
{
    pBuffer[i] = SPI1_ReadWriteByte(0xFF); /* 发送一个 0xFF (产生 SCK 时钟)，同时读取一个字节并存入缓冲区 */
}

GPIO_WriteBit(GPIOA, GPIO_Pin_2, 1); /* 拉高片选，结束本次通信 */
```

## 2.3 QSPI 进行写入

下面的代码用于 QSPI 通过单线向 FLASH 中写入的数据，其效果与后续的 SPI 写入一致。

```
/* 配置 QSPI 通信参数：指令、地址、数据均采用单线模式 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_IMode = QSPI_ComConfig_IMode_1Line;
QSPI_ComConfig_InitStructure.QSPI_ComConfig_ADMODE = QSPI_ComConfig_ADMODE_1Line;
QSPI_ComConfig_InitStructure.QSPI_ComConfig_DMODE = QSPI_ComConfig_DMODE_1Line;
QSPI_ComConfig_InitStructure.QSPI_ComConfig_FMODE = QSPI_ComConfig_FMODE_Indirect_Write; /* 间接写模式 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_ADSIZE = QSPI_ComConfig_ADSIZE_24bit; /* 地址宽度 24 位 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_INS = FLASH_CMD_PageProgram; /* 页编程指令 (0x02) */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_DummyCycles = 0; /* 无空指令周期 */
QSPI_ComConfig_Init(QSPI1, &QSPI_ComConfig_InitStructure);

QSPI_SetAddress(QSPI1, addr); /* 设置 24 位目标起始地址 */
QSPI_SetDataLength(QSPI1, len); /* 设置待写入数据长度 */

QSPI_DMAMCmd(QSPI1, ENABLE); /* 使能 QSPI 的 DMA 功能 */
QSPI_Tx_DMA(data, len); /* 配置 DMA 从内存将 len 字节数据搬移到 QSPI 发送寄存器 */
DMA_Cmd(DMA2_Channel1, ENABLE); /* 使能 DMA 通道开始传输 */

QSPI_Start(QSPI1); /* 启动 QSPI 通信，开始发送指令+地址+数据 */

while (DMA_GetFlagStatus(DMA2, DMA_FLAG_TC1) == RESET) /* 等待 DMA 传输完成标志 */
{
}
```

上面的代码配置 QSPI 为单线模式、间接写模式，使用 24 位地址和页编程指令（0x02），无交替字节和空指令周期，初始化后设置目标地址和数据长度，使能 QSPI 的 DMA 功能并通过 DMA 将内存数据搬运到发送寄存器，随后启动 QSPI 传输，并循环等待 DMA 传输完成标志以确认数据已全部发送完毕。

下面为效果相似的 SPI 代码：

```
GPIO_WriteBit(GPIOA, GPIO_Pin_2, 0); /* 拉低片选 (nCS) */
SPI1_ReadWriteByte(FLASH_CMD_PageProgram); /* 发送页编程指令 (0x02) */
SPI1_ReadWriteByte((u8)((WriteAddr) >> 16)); /* 发送 24 位目标地址的高 8 位 */
SPI1_ReadWriteByte((u8)((WriteAddr) >> 8)); /* 发送 24 位目标地址的中间 8 位 */
SPI1_ReadWriteByte((u8)WriteAddr); /* 发送 24 位目标地址的低 8 位 */

for (i = 0; i < size; i++) /* 循环写入数据 */
{
    SPI1_ReadWriteByte(pBuffer[i]); /* 逐个字节将待写入数据发送给 FLASH */
}

GPIO_WriteBit(GPIOA, GPIO_Pin_2, 1); /* 拉高片选，结束通信，触发 FLASH 内部编程执行 */
```

## 2.4 QSPI 轮询状态

在实际使用 SPI 时，常需要用到轮询来检查外设状态——最典型的例子就是在对 SPI FLASH 做擦除或页写之后，反复读取状态寄存器判断忙等（BUSY）是否结束：

```
uint8_t sr = 0; /* 定义状态寄存器变量 */
do
{
    Delay_Ms(1); /* 延时 1ms */
    GPIO_WriteBit(GPIOA, GPIO_Pin_2, 0); /* 拉低片选 */
    SPI1_ReadWriteByte(FLASH_CMD_ReadStatus_REG1); /* 发送读取状态寄存器 1 指令 (0x05) */
    sr = SPI1_ReadWriteByte(0xff); /* 发送 0xFF 产生时钟，同时读取状态寄存器 1 的值 */
    GPIO_WriteBit(GPIOA, GPIO_Pin_2, 1); /* 拉高片选 */
} while ((sr & 0x01) == 0x01); /* 检查状态寄存器最低位 (BUSY 位)，若为 1 表示 FLASH 正忙，则循环等待 */
```

这种写法每轮都占用 CPU。QSPI 的自动轮询模式（Auto-polling Mode）就是为此设计的硬件化替代：软件只需一次性配置“去读哪条状态命令、检查哪几位、匹配到何值就认为完成”，之后 QSPI 硬件便按设定的间隔自动发起读帧、自判位、命中即置 SMF 标志（可选中断），整个过程不需要 CPU 反复介入：

```
/* 读 SR1(0x05, 单线)，掩码 0x01 只关注 BUSY 位 (PSMKR),
 * 匹配希望值 0x00 (即不再忙, PSMA), PMM_AND 全位匹配; 命中置 SMF. */
QSPI_AutoPollingMode_Config(QSPI1, 0x0000, 0x01, QSPI_PMM_AND);
QSPI_AutoPollingMode_StopCmd(QSPI1, ENABLE); /* APMS: 命中即停 */
QSPI_AutoPollingMode_SetInterval(QSPI1, 2500); /* 轮询间隔 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_Ins = FLASH_CMD_ReadStatus_REG1; /* 0x05 */
QSPI_ComConfig_InitStructure.QSPI_ComConfig_FMode = QSPI_ComConfig_FMode_Auto_Polling;
QSPI_Start(QSPI1);
```



```
while (QSPI_GetFlagStatus(QSPI1, QSPI_FLAG_SM) == RESET) /* 等状态匹配 */  
    ;
```

历史版本

| 日期       | 版本   | 变更内容 |
|----------|------|------|
| 2026/7/8 | V1.0 | 初版发行 |

## 声明

本手册仅供参考。作者在不另行通知的情况下，可随时进行修改。